

Лабораторная работа: построение и визуализация фракталов

Множества Мандельброта, Жюлиа и ковёр Серпинского

Евграфов Артём, Гузалов Тимур

13 октября 2025 г.

1. Свойства множества Мандельброта: доказательства

Рассматривается рекуррентная последовательность

$$z_{n+1} = z_n^2 + c, \quad z_0 = 0, \quad c \in \mathbb{C}.$$

Множество Мандельброта M состоит из всех параметров c , для которых последовательность $\{z_n\}$ ограничена.

Утверждение 1 (Симметрия относительно вещественной оси). *Если $c \in M$, то $\bar{c} \in M$, то есть M инвариантно относительно комплексного сопряжения.*

Доказательство. Пусть $z_{n+1} = z_n^2 + c$, $z_0 = 0$, и последовательность $\{|z_n|\}$ ограничена. Рассмотрим $k_n = \bar{z}_n$. Тогда

$$k_{n+1} = \overline{z_{n+1}} = \overline{z_n^2 + c} = \overline{z_n}^2 + \bar{c} = k_n^2 + \bar{c}, \quad k_0 = \bar{z}_0 = 0,$$

то есть для всех n имеем $|k_n| = |z_n|$, следовательно (k_n) ограничена. Значит, $\bar{c} \in M$. $\square$

Утверждение 2 (Критерий исключения при больших модулях). *Если $|c| > 2$, то $c \notin M$.*

Доказательство. Хотим доказать, что при $|c| > 2$, $\lim_{n \rightarrow \infty} |z_n| = \infty$, $z_1 = c$, $|z_1| = |c|$
 $z_2 = c^2 + c$, $|z_2| = |(c^2 + c)|$

Вспомним неравенство: $|a + b| \geq ||a| - |b||$

Отсюда $|z_2| = |(c^2 + c)| \geq |c^2| - |c| = |c|(|c| - 1)$

$z_3 = (c^2 + c)^2 + c$, $|z_3| = |(c^2 + c)^2 + c| \geq |c^2 + c|^2 - |c| \geq (|c^2| - |c|)^2 - |c| = |c|((|c|^2 - |c|)(|c| - 1) - 1)$
 $|c|(|c| - 1)^2$. Поверим в то, что $|z_n| \geq |c|(|c| - 1)^{n-1}$ и докажем это по индукции.

База очевидна, сделаем индукционный переход. Пусть предположение верно, в таком случае докажем, что $z_{n+1} \geq |c|(|c| - 1)^n$.

$|z_{n+1}| = |z_n^2 + c| \geq |z_n|^2 - |c| = |z_n|^2 - |c| = (|c|(|c| - 1)^{n-1})^2 - |c|$.

Докажем, что $(|c|(|c| - 1)^{n-1})^2 - |c| > |c|(|c| - 1)^n$

$|c|(|c| - 1)^{2n-2} - 1 > (|c| - 1)^n$

$$|c|(|c| - 1)^{2n-2} - 1 > (|c| - 1)^n$$

Пусть $l = |c| - 1$, тогда $|c| = l + 1$

$$(l + 1)l^{2n-2} - 1 > l^n$$

$$l^{2n-1} + l^{2n-2} - 1 \geq l^n$$

$$l^{n-1} + l^{n-2} - \frac{1}{l^n} \geq 1 \text{ так как } |c| > 2, l > 1.$$

Отсюда получаем, что $z_{n+1} \geq |c|(|c| - 1)^n$. Заметим, что $\lim_{n \rightarrow \infty} (|c| - 1)^n = \infty$ при

$|c| > 2$. □

2. Код визуализации (Python)

Скрипт строит изображения множеств Мандельброта, Жюлиа и ковра Серпинского.

```
import numpy as np
import matplotlib.pyplot as plt
from pathlib import Path

def make_plane(center=0+0j, height=3.0, width_px=1200, height_px=900):
    aspect = width_px / height_px          # соотношение
    сторон изображения
    half_w = (height * aspect) / 2.0      # половина ширины
    окна на C
    half_h = height / 2.0                 # половина высоты
    окна на C
    xs = np.linspace(center.real - half_w,
                      center.real + half_w, width_px) # равномерная
    сетка по Re
    ys = np.linspace(center.imag - half_h,
                      center.imag + half_h, height_px) # равномерная
    сетка по Im
    return xs[None, :] + 1j * ys[:, None]    # формируем 2Dполе-
    комплексных точек

def mandelbrot_escape_times(center=-0.5+0j, height=3.0, width_px=1200,
                             height_px=900,
                             max_iter=300, escape_radius=2.0):

    C = make_plane(center, height, width_px, height_px) # каждая точка C -
    параметр в  $z_{n+1} = z_n^2 + C$ 
    Z = np.zeros_like(C, dtype=np.complex128)          # начальное  $z_0 = 0$ 
    для всех пикселей
    escaped_at = np.zeros(C.shape, dtype=np.int32)      # сюда пишем номер
    итерации, когда точка убежала
    mask = np.ones(C.shape, dtype=bool)                # маска ещё"
    активных" пикселей
    R2 = float(escape_radius * escape_radius)           # работаем с  $|z|^2$ 
    чтобы избежать sqrt
```

```

    for i in range(max_iter):
        Z[mask] = Z[mask] * Z[mask] + C[mask]          #  $z \leftarrow z^2 + c$ ,
        только на ещё активных точках
        m_esc = (Z.real*Z.real + Z.imag*Z.imag > R2) & mask
        escaped_at[m_esc] = i + 1                        # фиксируем момент
        побега
        mask[m_esc] = False                             # больше эти точки
        не обновляем
        if not mask.any():                             # если все
        сбежали/зафиксированы/ – можно остановиться
            break
    return escaped_at, (-height*width_px/height_px/2.0 + center.real,
                      +height*width_px/height_px/2.0 + center.real,
                      center.imag - height/2.0, center.imag + height/2.0)

def render_mandelbrot_runs(runs, outdir="mandelbrot_out"):
    Path(outdir).mkdir(parents=True, exist_ok=True)
    for k, cfg in enumerate(runs, 1):
        esc, extent = mandelbrot_escape_times(**cfg)
        plt.figure()
        plt.imshow(esc, extent=extent, origin='lower', interpolation='
nearest')
        plt.xlabel('Re(c)'); plt.ylabel('Im(c)')
        title = f"Мандельброт: итерации={cfg['max_iter']}"
        plt.title(title)
        plt.tight_layout()
        fname = f"{outdir}/mandelbrot_{k:02d}.png"
        plt.savefig(fname, dpi=200)
        plt.close()
        print("Сохранено:", fname)

def julia_escape_times(c=-0.4+0.6j, center=0+0j, height=3.2, width_px=1200,
                      height_px=1200,
                      max_iter=300, escape_radius=2.0):

    Z = make_plane(center, height, width_px, height_px) # начальные точки
    Z0 – это координаты пикселей
    escaped_at = np.zeros(Z.shape, dtype=np.int32)
    mask = np.ones(Z.shape, dtype=bool)                # обновляем только
    ещё живые"" точки
    R2 = float(escape_radius * escape_radius)

    for i in range(max_iter):
        Z[mask] = Z[mask] * Z[mask] + c                # фиксированный
        параметр c
        m_esc = (Z.real*Z.real + Z.imag*Z.imag > R2) & mask
        escaped_at[m_esc] = i + 1
        mask[m_esc] = False
        if not mask.any():
            break

```

```

    aspect = width_px / height_px
    half_w = (height * aspect) / 2.0
    half_h = height / 2.0
    extent = (center.real - half_w, center.real + half_w, center.imag -
half_h, center.imag + half_h)
    return escaped_at, extent

def render_julia_runs(runs, outdir="julia_out"):
    Path(outdir).mkdir(parents=True, exist_ok=True)
    for k, cfg in enumerate(runs, 1):
        esc, extent = julia_escape_times(**cfg)
        plt.figure()
        plt.imshow(esc, extent=extent, origin='lower', interpolation='
nearest')
        plt.xlabel('Re(z)'); plt.ylabel('Im(z)')
        title = f"Жюлиа c={cfg['c']}, итерации={cfg['max_iter']}"
        plt.title(title)
        plt.tight_layout()
        fname = f"{outdir}/julia_{k:02d}.png"
        plt.savefig(fname, dpi=200)
        plt.close()
        print("охраненоC:", fname)

def sierpinski_carpet_mask(depth=4):
    N = 3 ** depth
    img = np.ones((N, N), dtype=np.uint8)
    for k in range(depth):
        blk = 3 ** (depth - 1 - k)
        step = 3 * blk
        for i in range(blk, N, step):
            for j in range(blk, N, step):
                img[i:i+blk, j:j+blk] = 0
    return img

def render_carpet_runs(depths, scale=2, outdir="carpet_out"):
    Path(outdir).mkdir(parents=True, exist_ok=True)
    for d in depths:
        img = sierpinski_carpet_mask(d)
        plt.figure(figsize=(scale*4, scale*4))
        plt.imshow(img, cmap='gray', origin='lower', interpolation='nearest
') # не задаём цвета явно
        plt.axis('off')
        plt.title(f"Ковёр Серпинского глубина(={d}, размер={img.shape[0]}x{
img.shape[1]})")
        fname = f"{outdir}/carpet_depth_{d}.png"
        plt.tight_layout(pad=0)
        plt.savefig(fname, dpi=200, bbox_inches='tight', pad_inches=0)
        plt.close()
        print("Сохранено:", fname)

```

```

if __name__ == "__main__":
    mb_runs = [
        dict(center=-0.5+0j, height=3.0, width_px=1200, height_px=900,
max_iter=60),
        dict(center=-0.75+0.1j, height=0.6, width_px=1400, height_px=900,
max_iter=250),
        dict(center=-0.743+0.131j, height=0.05, width_px=1400, height_px
=900, max_iter=600),
        dict(center=-0.743643887037151+0.131825904205330j, height=0.005,
width_px=1600, height_px=1000, max_iter=1500),
    ]
    render_mandelbrot_runs(mb_runs)

    jl_runs = [
        dict(c=-0.4+0.6j, center=0+0j, height=3.2, width_px=1200,
height_px=1200, max_iter=200),
        dict(c=-0.70176-0.3842j, center=0+0j, height=2.4, width_px=1400,
height_px=1400, max_iter=500),
        dict(c=-0.835-0.2321j, center=0+0j, height=1.6, width_px=1600,
height_px=1600, max_iter=1000),
    ]
    render_julia_runs(jl_runs)

    render_carpet_runs(depths=[2, 3, 4, 5, 6], scale=2)

```

3. Места для изображений

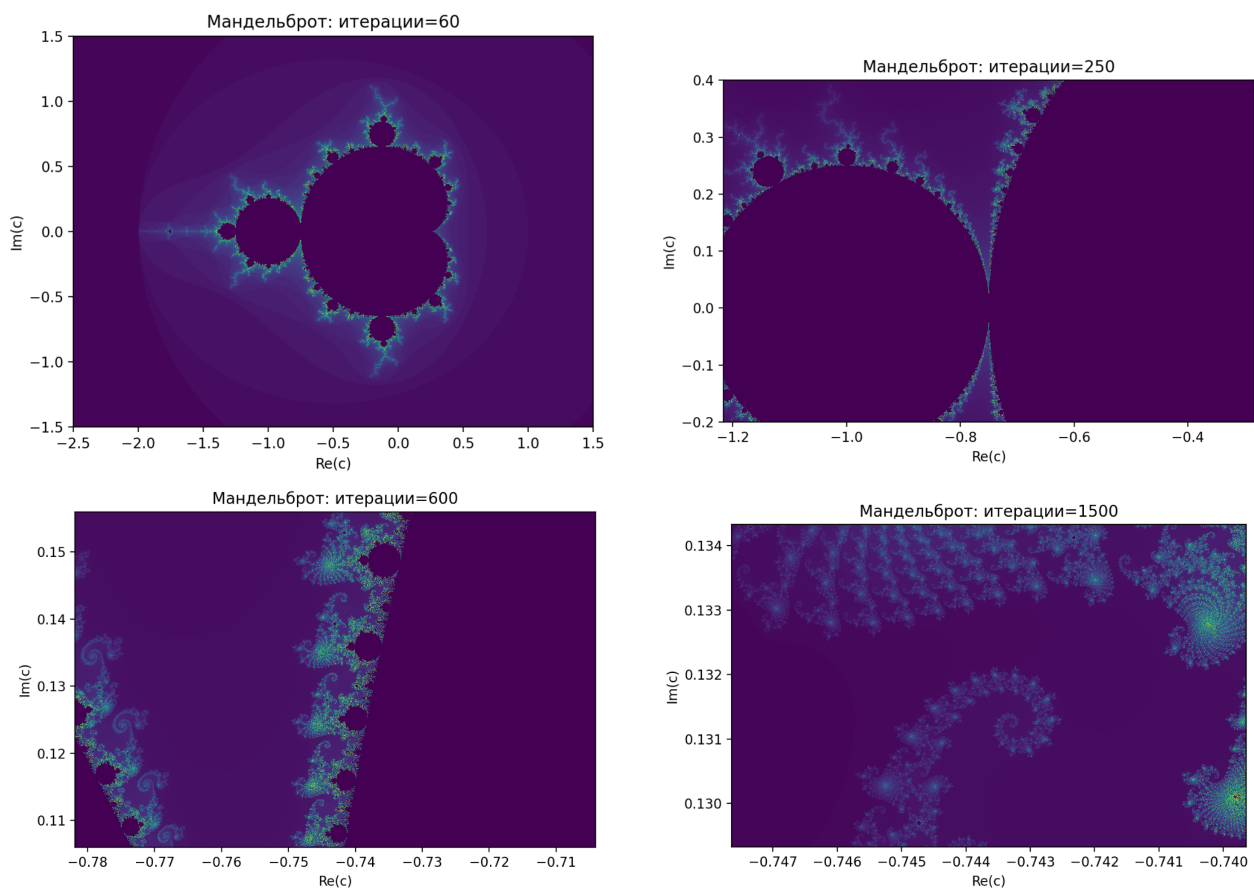


Рис. 1: Множество Мандельброта при разных итерациях и приближениях.

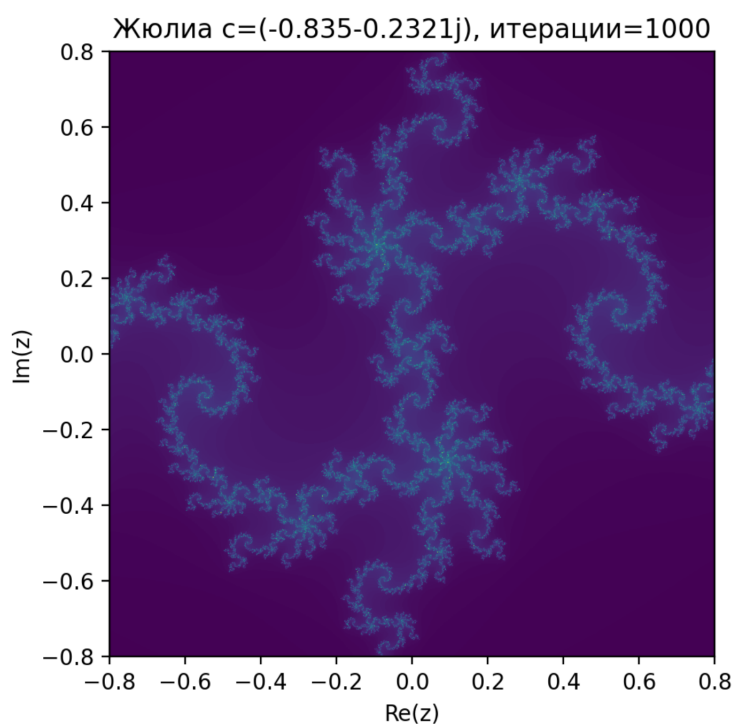
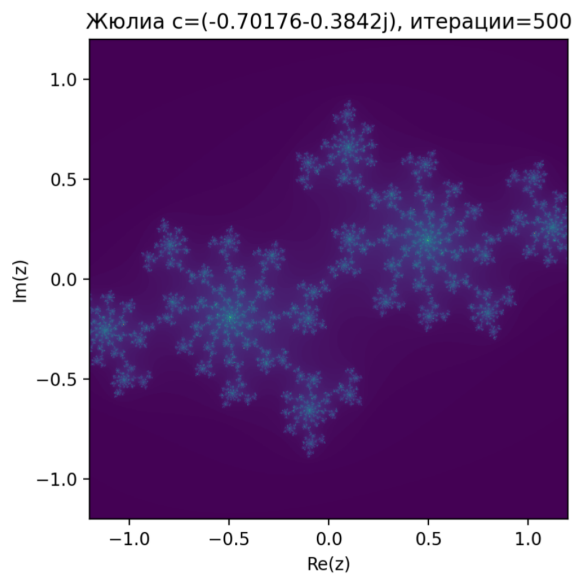
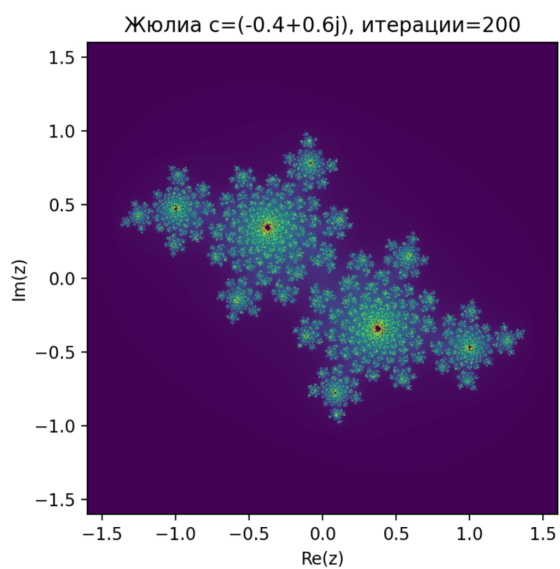
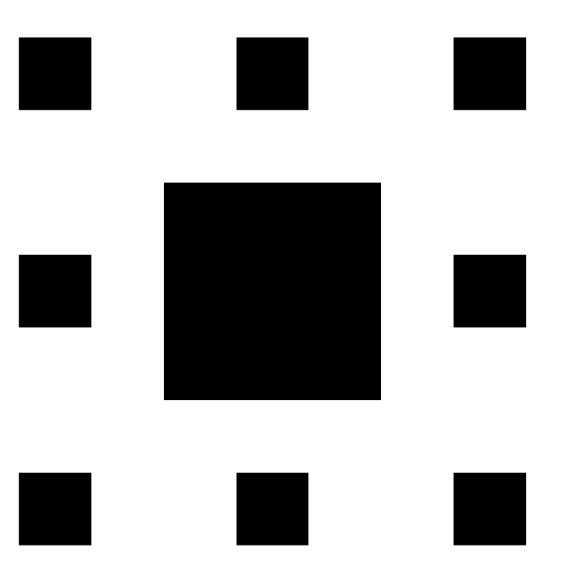
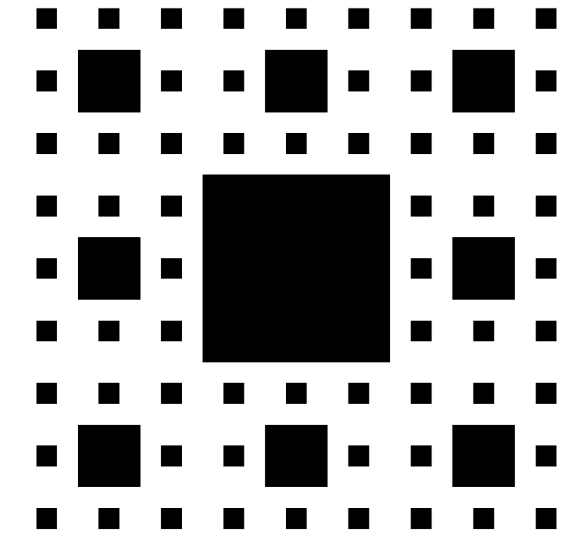


Рис. 2: Заполненные множества Жюлиа для разных параметров c и масштабов.

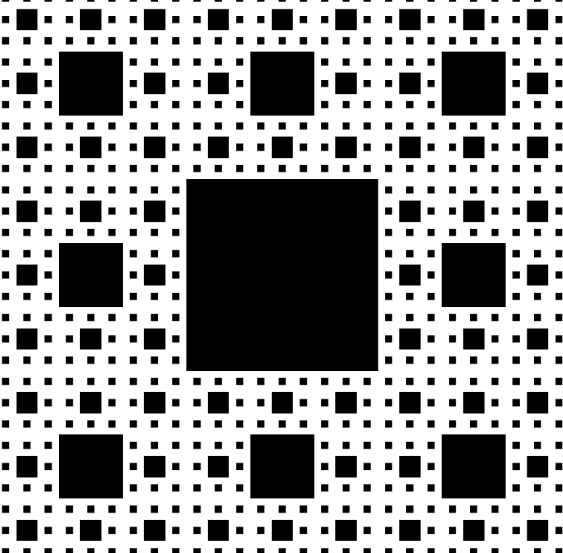
Ковёр Серпинского (глубина=2, размер=9x9)



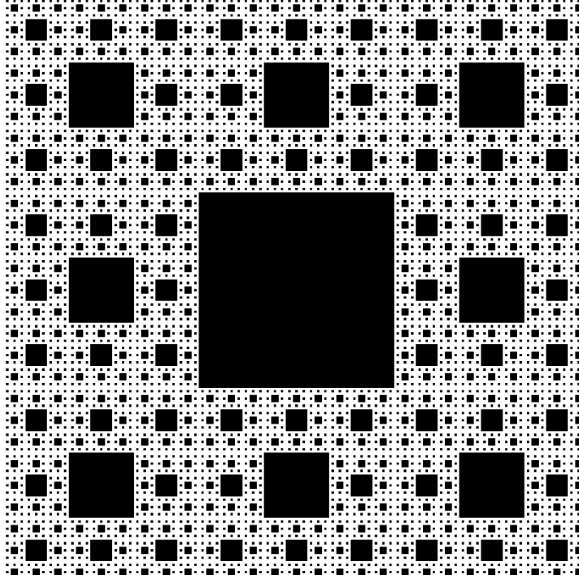
Ковёр Серпинского (глубина=3, размер=27x27)



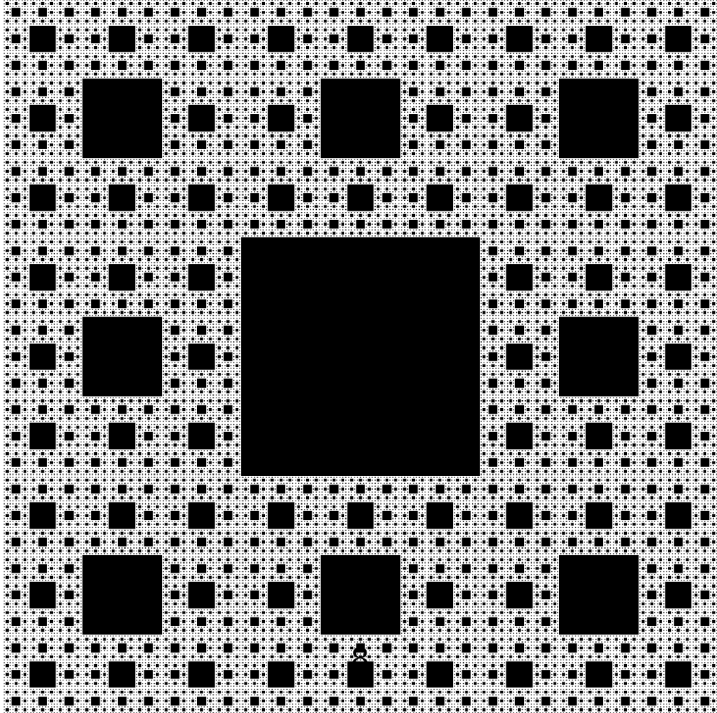
Ковёр Серпинского (глубина=4, размер=81x81)



Ковёр Серпинского (глубина=5, размер=243x243)



Ковёр Серпинского (глубина=6, размер=729x729)



4. Текст-описание структуры и построения ковра Серпинского

Определение и самоподобие. Ковёр Серпинского — плоский самоподобный фрактал, получающийся из единичного квадрата последовательным удалением центральных квадратов при разбиении на сетку 3×3 . На каждом шаге остаётся 8 копий, подобных исходной.

Детерминированное построение. Пусть $Q_0 = [0, 1]^2$. Разбиваем Q_0 на сетку 3×3 и удаляем центральный квадрат. К каждому из 8 оставшихся квадратов применяем ту же операцию и повторяем бесконечно. После n шагов: число ячеек $= 8^n$, длина стороны ячейки $= 3^{-n}$, суммарная площадь $= (8/9)^n \rightarrow 0$.

Критерий принадлежности Точка $(x, y) \in [0, 1]^2$ принадлежит коврику тогда и только тогда, когда в троичной записи $x = \sum x_k 3^{-k}$, $y = \sum y_k 3^{-k}$ никогда не встречается позиция с $x_k = y_k = 1$ одновременно.